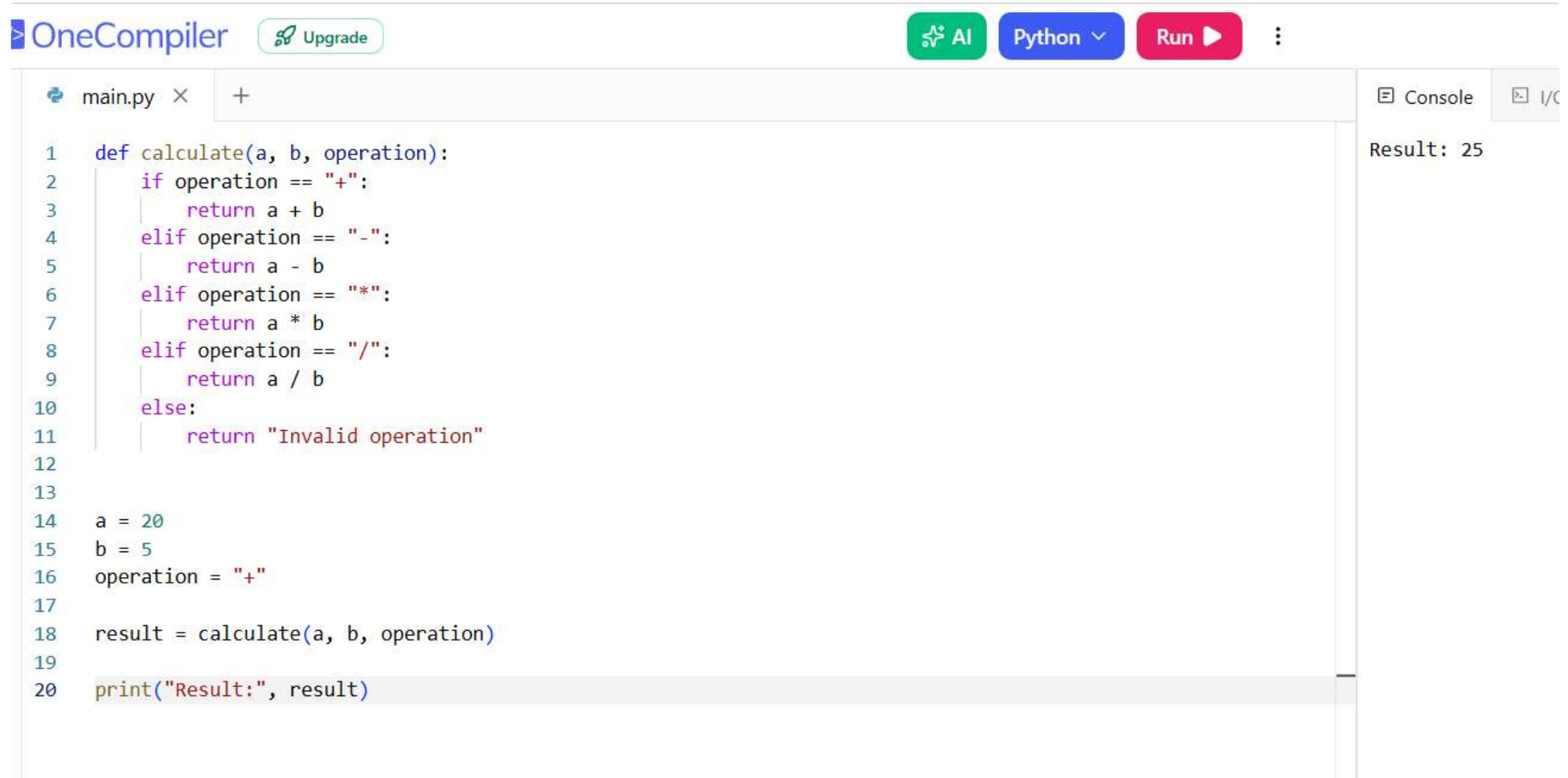


1. Write a function calculate(a, b, operation) that performs addition, subtraction, multiplication, or division based on the supplied operation.

```
def calculate(a, b, operation):  
    if operation == "+":  
        return a + b  
    elif operation == "-":  
        return a - b  
    elif operation == "*":  
        return a * b  
    elif operation == "/":  
        return a / b  
    else:  
        return "Invalid operation"  
  
a = 20  
b = 5  
operation = "+"  
  
result = calculate(a, b, operation)  
  
print("Result:", result)
```

output:



The screenshot shows the OneCompiler Python IDE interface. At the top, there's a header with the OneCompiler logo, an 'Upgrade' button, and buttons for 'AI', 'Python', 'Run', and a menu icon. Below the header, the editor shows a file named 'main.py' with the following Python code:

```
1 def calculate(a, b, operation):
2     if operation == "+":
3         return a + b
4     elif operation == "-":
5         return a - b
6     elif operation == "*":
7         return a * b
8     elif operation == "/":
9         return a / b
10    else:
11        return "Invalid operation"
12
13
14 a = 20
15 b = 5
16 operation = "+"
17
18 result = calculate(a, b, operation)
19
20 print("Result:", result)
```

On the right side, there's a 'Console' tab showing the output: 'Result: 25'.

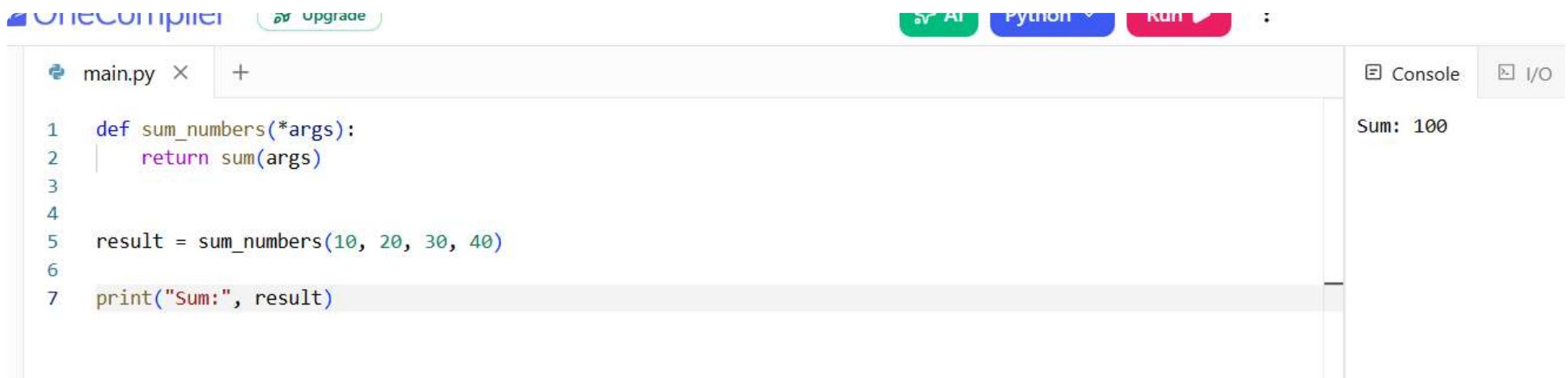
2. Write a function `sum_numbers(*args)` that accepts any number of arguments and returns their sum.

```
def sum_numbers(*args):
    return sum(args)
```

```
result = sum_numbers(10, 20, 30, 40)

print("Sum:", result)
```

output:



The screenshot shows the OneCompiler Python IDE interface. At the top, there's a header with the OneCompiler logo, an 'Upgrade' button, and tabs for 'AI', 'Python', and 'Run'. Below the header, a file named 'main.py' is open in the editor. The code in the editor is as follows:

```
1 def sum_numbers(*args):
2     return sum(args)
3
4
5 result = sum_numbers(10, 20, 30, 40)
6
7 print("Sum:", result)
```

On the right side of the editor, there are two tabs: 'Console' and 'I/O'. The 'Console' tab is active, displaying the output of the program: 'Sum: 100'.

3. Write a function `employee(**args)` that accepts employee information such as name, ID, department and salary, then displays the information.

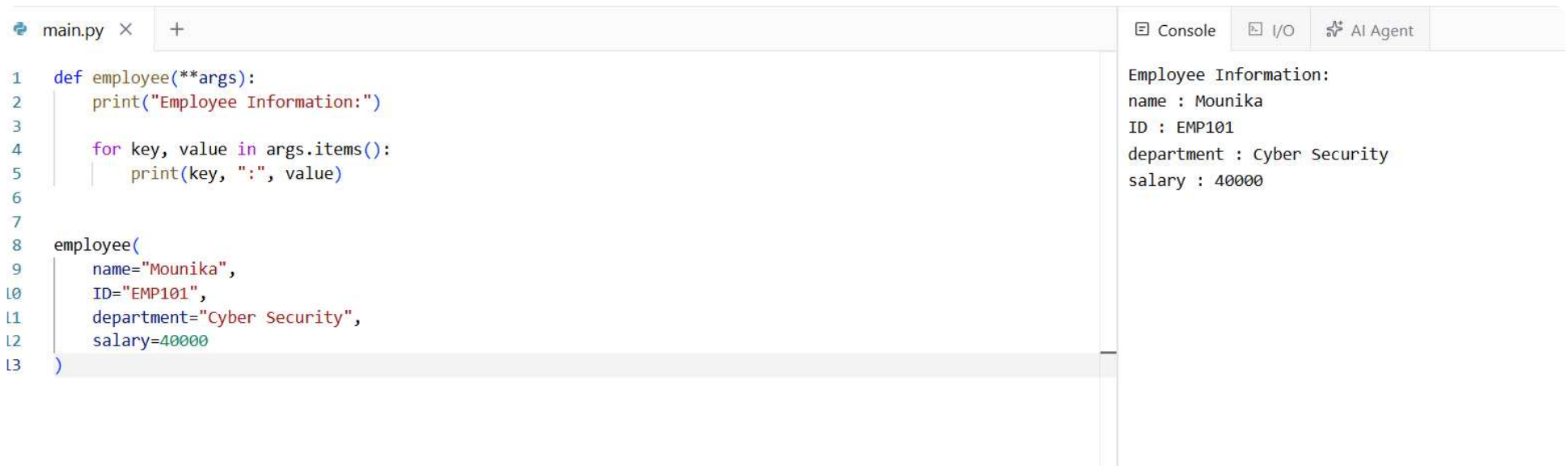
```
def employee(**args):
    print("Employee Information:")

    for key, value in args.items():
```

```
print(key, ":", value)
```

```
employee(  
    name="Mounika",  
    ID="EMP101",  
    department="Cyber Security",  
    salary=40000  
)
```

output:



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'employee' that takes arbitrary keyword arguments and prints them in a formatted way. It then calls the 'employee' function with specific values for name, ID, department, and salary. To the right of the code editor, there is a 'Console' panel showing the output of the program.

```
main.py × +  
1 def employee(**args):  
2     print("Employee Information:")  
3  
4     for key, value in args.items():  
5         print(key, ":", value)  
6  
7  
8 employee(  
9     name="Mounika",  
10    ID="EMP101",  
11    department="Cyber Security",  
12    salary=40000  
13 )
```

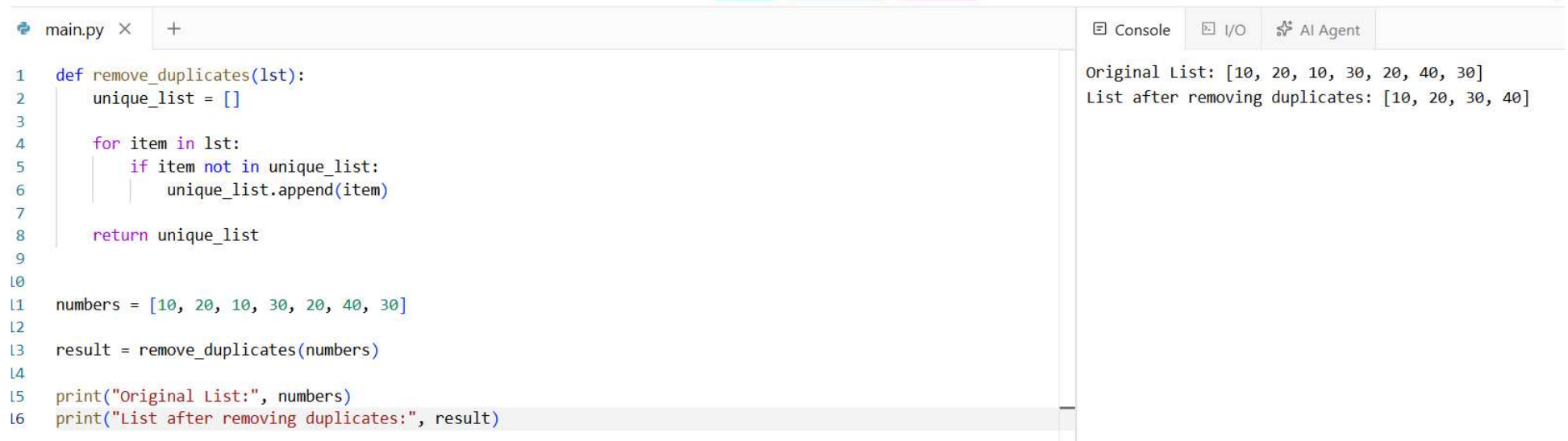
Console I/O AI Agent

Employee Information:
name : Mounika
ID : EMP101
department : Cyber Security
salary : 40000

4. Write a function `remove_duplicates(lst)` that returns a list containing only unique elements while preserving their original order.

```
def remove_duplicates(lst):  
    unique_list = []  
  
    for item in lst:  
        if item not in unique_list:  
            unique_list.append(item)  
  
    return unique_list  
  
numbers = [10, 20, 10, 30, 20, 40, 30]  
  
result = remove_duplicates(numbers)  
  
print("Original List:", numbers)  
print("List after removing duplicates:", result)
```

output:



```
main.py x +  
1 def remove_duplicates(lst):  
2     unique_list = []  
3  
4     for item in lst:  
5         if item not in unique_list:  
6             unique_list.append(item)  
7  
8     return unique_list  
9  
10  
11 numbers = [10, 20, 10, 30, 20, 40, 30]  
12  
13 result = remove_duplicates(numbers)  
14  
15 print("Original List:", numbers)  
16 print("List after removing duplicates:", result)
```

Console

I/O

AI Agent

Original List: [10, 20, 10, 30, 20, 40, 30]
List after removing duplicates: [10, 20, 30, 40]

5. Using a lambda function, sort a list of tuples based on the second element. Example: [(1,5), (2,3), (4,1)].

```
numbers = [(1, 5), (2, 3), (4, 1)]
```

```
sorted_list = sorted(numbers, key=lambda x: x[1])
```

```
print("Original List:", numbers)
```

```
print("Sorted List:", sorted_list)
```

output:

```
main.py x +  
1 numbers = [(1, 5), (2, 3), (4, 1)]  
2  
3 sorted_list = sorted(numbers, key=lambda x: x[1])  
4  
5 print("Original List:", numbers)  
6 print("Sorted List:", sorted_list)
```

Console I/O AI Agent

Original List: [(1, 5), (2, 3), (4, 1)]
Sorted List: [(4, 1), (2, 3), (1, 5)]